

Высокоуровневый язык программирования

Высокоуровневый язык программирования — [язык программирования](#), разработанный для быстроты и удобства использования [программистом](#). Основная черта высокоуровневых языков — это [абстракция](#), то есть введение смысловых конструкций, кратко описывающих такие [структуры данных](#) и [операции над ними](#), описания которых на [машинном коде](#) (или другом [низкоуровневом языке программирования](#)) очень длинны и сложны для понимания.

Высокоуровневые языки программирования были разработаны для платформенной независимости сути алгоритмов. Зависимость от платформы перекладывается на инструментальные программы — [трансляторы](#), компилирующие текст, написанный на языке высокого уровня, в элементарные машинные команды (инструкции). Поэтому, для каждой платформы разрабатывается платформенно-уникальный транслятор для каждого высокоуровневого языка, например, переводящий текст, написанный на [Delphi](#) в элементарные команды [микропроцессоров](#) семейства [x86](#).

Так, высокоуровневые языки стремятся не только облегчить решение сложных программных задач, но и упростить [портирование программного обеспечения](#). Использование разнообразных [трансляторов](#) и [интерпретаторов](#) обеспечивает связь [программ](#), написанных при помощи языков высокого уровня, с различными [операционными системами](#) программируемыми устройствами и оборудованием, и, в идеале, не требует модификации [исходного кода](#) (текста, написанного на высокоуровневом языке) для любой платформы.

Такого рода оторванность высокоуровневых языков от аппаратной реализации [компьютера](#) помимо множества плюсов имеет и минусы. В частности, она не позволяет создавать простые и точные инструкции к используемому оборудованию. Программы, написанные на языках высокого уровня, проще для понимания программистом, но менее эффективны, чем их аналоги, создаваемые при помощи [низкоуровневых языков](#). Одним из следствий этого стало добавление поддержки того или иного языка низкого уровня ([язык ассемблера](#)) в ряд современных профессиональных высокоуровневых языков программирования.

Примеры: [C \(Си\)](#), [C++](#), [Delphi](#), [Fortran](#), [Java](#), [JavaScript](#), [Лисп](#), [PHP](#), также [Python](#), [Ruby](#), [Perl](#), которые иногда называют [сверхвысокоуровневыми](#). Языкам высокого уровня свойственно умение работать с комплексными структурами данных. В большинстве из них интегрирована поддержка [строковых типов](#), [объектов](#), операций [файлового ввода-вывода](#) и т. п.

Первым языком программирования высокого уровня считается [компьютерный язык Plankalkül](#), разработанный немецким инженером [Конрадом Цузе](#) ещё в период [1942—1946 годов](#). Однако транслятора для него не существовало до 2000 года. Первым в мире транслятором языка высокого уровня является ПП (Программирующая Программа), он же ПП-1, успешно испытанный в 1954 году в СССР. Транслятор ПП-2 (1955 год, 4-й в мире транслятор) уже был оптимизирующим и содержал собственный загрузчик и отладчик, библиотеку стандартных процедур, а транслятор ПП для [ЭВМ Стрела-4](#) уже содержал и компоновщик (linker) из модулей. Однако, широкое применение высокоуровневых языков началось с возникновением [Фортрана](#) и созданием [компилятора](#) для этого языка ([1957](#)).

На 2023 год по версии компании TIOBE Software лидирует язык программирования [Python](#).

Переносимость программ



Распространено мнение, что [программы](#) на языках высокого уровня можно написать один раз и потом использовать на [компьютере](#) любого типа. В действительности же это верно только для тех программ, которые мало взаимодействуют с [операционной системой](#), например, выполняют какие-либо вычисления или обработку данных. Большинство же интерактивных (а тем более мультимедийных) программ обращаются к [системным вызовам](#), которые сильно различаются в зависимости от операционной системы. Например, для отображения графики на экране компьютера программы под [Microsoft Windows](#) используют функции [Windows API](#), которые отличаются от используемых в системах, поддерживающих стандарт [POSIX](#). Чаще всего для этих целей в них используется [программный интерфейс X-сервера](#).

К настоящему времени создан целый ряд [программных библиотек](#) (например, библиотека [Qt](#) или [wxWidgets](#)), скрывающих несоответствия системных вызовов различных операционных систем от [прикладных программ](#). Однако такие библиотеки, как правило, не позволяют полностью использовать все возможности конкретных операционных систем.

Новые тенденции

Новой тенденцией является появление языков программирования немного более высокого уровня (ультравысокоуровневых, не путать со [сверхвысокоуровневыми](#)), [графических языков](#), так называемых языков четвертого поколения ([4GL](#)). Такого рода языки характеризуются наличием дополнительных структур и объектов, ориентированных на прикладное использование. Прикладные объекты, в свою очередь, требуют минимальной настройки в виде параметров и моментально готовы к использованию. Использование ультравысокоуровневых языков программирования снижает временные

затраты на разработку программного обеспечения и повышает качество конечного продукта за счёт, опять-таки, уменьшения объёма исходных кодов.

См. также

- [Сверхвысокоуровневый язык программирования](#)
- [Низкоуровневый язык программирования](#)
- [Метапрограммирование](#)
- [Псевдокод \(язык описания алгоритмов\)](#)

У этой статьи есть 3 проблемы, помогите их исправить:

- В статье не хватает ссылок на источники (см. рекомендации по поиску).
- В статье есть список источников, но не хватает сносок.

[\[скрыть\]](#)

[Узнать больше](#)

[Узнать больше](#)

[Сообщить об ошибке](#)

